

UNMANNED STORE MANAGER

무인편의점 관리자

</> C LANGUAGE

CHOICE-BASED SIMULATION

ALGORITHM PRACTICE

COURSE

C언어 알고리즘 실습 · 기말 프로젝트

PRESENTER

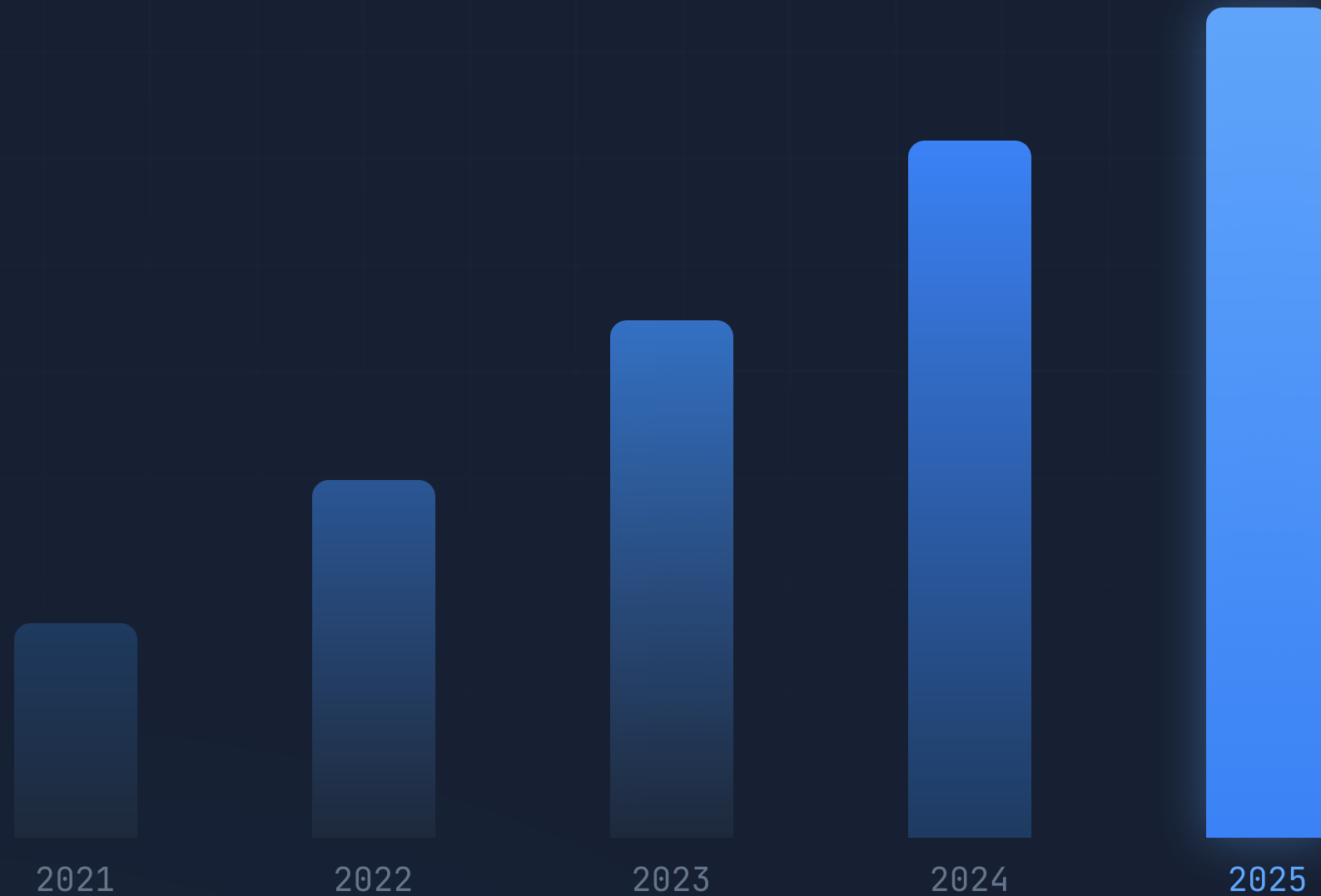
박수용 2601110288

배경 및 동기

무인 매장은 빠르게 늘고 있지만, 사람이 없는 만큼 보안과 운영 리스크는 점장의 판단에 달려 있습니다.

무인 매장 증가 추세 • TREND

▲ 지속 증가



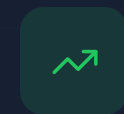
PROBLEM • 문제

무인 운영의 보안 취약성과 절도 위험



CAUSE • 원인

인건비 상승과 무인화 확산



OPPORTUNITY • 기회

운영 판단을 게임으로 학습

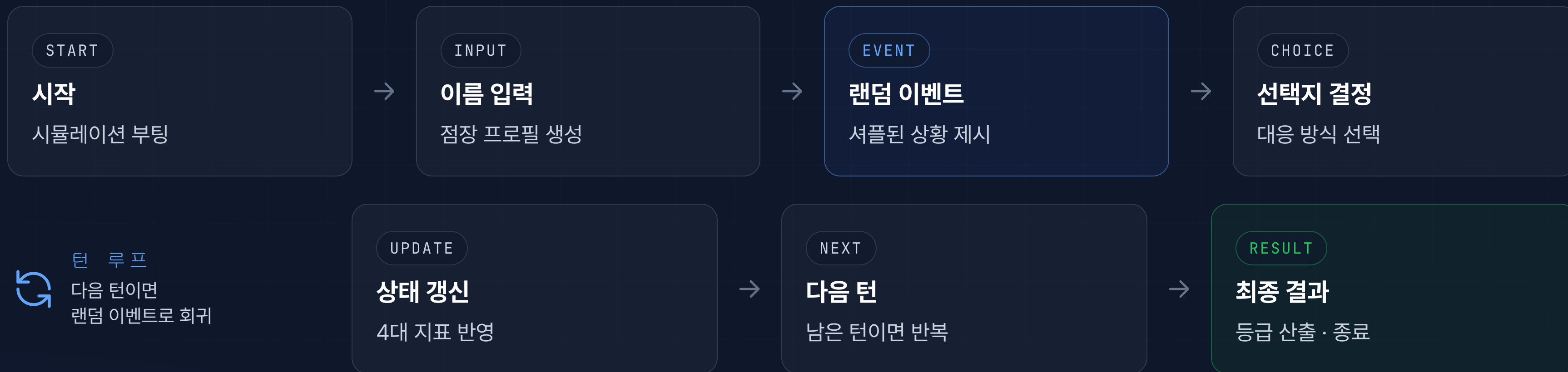
프로젝트 개요

플레이어는 무인편의점 점장이 되어 **CCTV로 매장을 감시**하고, 발생하는 상황에 선택으로 대응합니다. 모든 결정은 매장의 상태 지표를 바꾸고, 마지막에 최종 평가로 이어집니다.

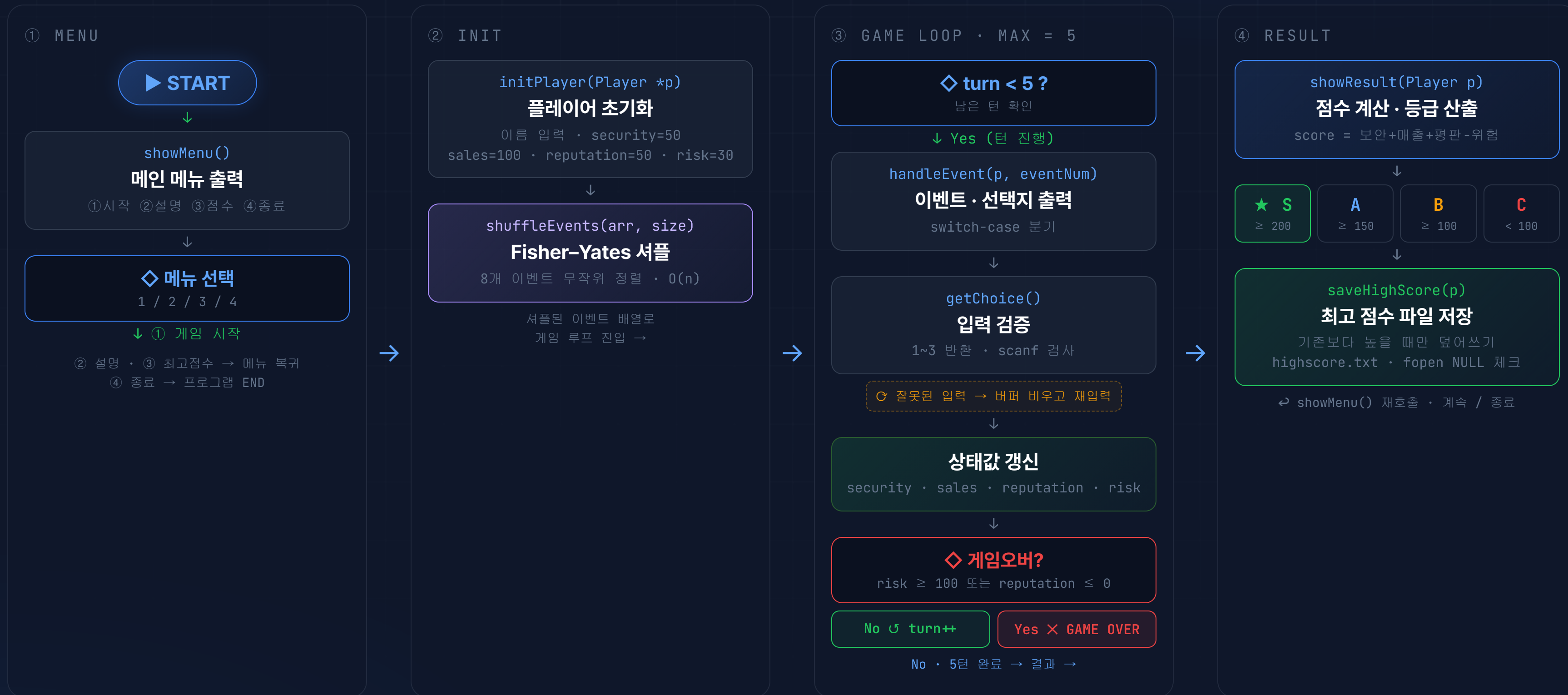


게임 흐름

시작부터 결과까지 하나의 턴 루프로 진행됩니다. 매 턴 이벤트가 반복되며 상태가 누적됩니다.



게임 플로우차트



상태 관리 시스템

모든 선택은 네 가지 핵심 지표로 환산됩니다. 균형을 유지할수록 높은 최종 등급에 가까워집니다.



이벤트 시스템

매 턴 발생하는 상황은 세 가지 범주로 구성됩니다. 어떤 범주가 나올지는 셔플로 무작위 결정됩니다.

SECURITY 보안 이벤트

- 절도 의심 행동 포착
- 수상한 심야 방문자
- CCTV 사각지대 발생
- 출입문 강제 개방 시도

▶ 보안 점수 • 절도 위험에 영향

CUSTOMER 고객 서비스 이벤트

- 고객 불만 접수
- 결제기 오류 발생
- 환불·교환 요청
- 상품 위치 문의

▶ 매출 • 평판에 영향

MANAGEMENT 매장 관리 이벤트

- 냉장고 고장
- 인기 상품 재고 부족
- 매장 청결 상태 저하
- 정전·설비 점검 알림

▶ 평판 • 매출에 영향

프로그램 구조

플레이어의 모든 상태는 하나의 구조체 **Player**로 묶여 관리됩니다.

player.h

```
// 점장의 상태를 담는 구조체
typedef struct {
    char  name[20]; // 이름
    int   security; // 보안 점수
    int   sales;    // 매출
    int   reputation; // 평판
    int   risk;     // 절도 위험
    int   score;    // 종합 점수
} Player;
```

▶ Player

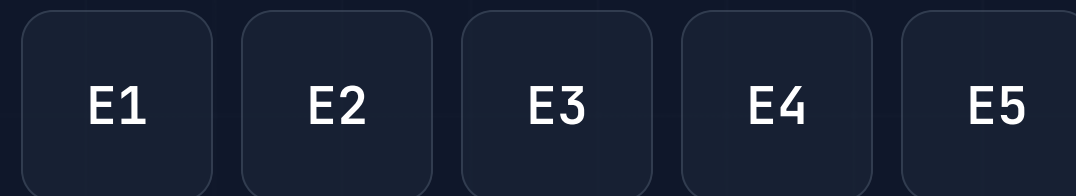
```
├— name char[20]
├— security int · 보안 점수
├— sales int · 매출
├— reputation int · 평판
├— risk int · 절도 위험
└— score int · 종합 점수
```

하나의 구조체로 묶어 함수 간 전달과 갱신을 일관되게 처리합니다.

핵심 알고리즘 Fisher-Yates Shuffle

이벤트 순서를 매 게임마다 무작위로 섞어, 같은 패턴이 반복되지 않게 합니다.

① ORIGINAL ORDER · 원본 순서



↓ $i = n-1 \rightarrow 1$ · swap(i , rand 0.. i)

② SHUFFLED ORDER · 무작위 순서



shuffle.c

```
void shuffle(int a[], int n) {  
    for (int i = n-1; i > 0; i--) {  
        int j = rand() % (i+1);  
        // a[i] ↔ a[j] 교환  
        int t = a[i];  
        a[i] = a[j];  
        a[j] = t;  
    }  
}
```

시간복잡도 $O(n)$ · 모든 배열을 한 번씩만 순회

사용 기술

수업에서 다룬 C언어 핵심 요소를 실제 게임 로직에 빠짐없이 적용했습니다.



ARRAYS

배열

이벤트·선택지 목록 저장



STRUCTURES

구조체

Player 상태 통합 관리



POINTERS

포인터

구조체를 함수로 전달·갱신



FILE I/O

파일 입출력

최종 결과 기록·저장



LOOPS

반복문

턴 루프 구동



CONDITIONS

조건문

선택 분기·등급 판정



RANDOM

난수 함수

rand·srand 이벤트 셔플



MACROS

매크로

#define 상수·설정 정의

어려웠던 점 및 해결 방법

구현 과정에서 부딪힌 세 가지 문제와, 이를 해결하며 얻은 핵심 교훈입니다.

01 Fisher-Yates 셔플 구현

■ 문제

`rand()`만 사용하면 중복 이벤트가 발생

■ 해결

배열 전체를 셔플한 뒤 앞 5개만 사용

■ 핵심

`rand()%i` 와 `rand()%(i+1)` 차이로 마지막 원소 고정 버그 발생 → `i+1`로 수정

02 구조체 포인터와 화살표 연산자

■ 문제

`Player` 구조체를 값으로 전달하면 `main()`에 변경이 미반영

■ 해결

`Player *p` 포인터로 전달, `->` 연산자로 직접 접근

■ 핵심

값 전달 vs 포인터 전달의 차이를 직접 체감

03 입력 버퍼 처리 문제

■ 문제

문자 입력 시 `scanf` 실패 + 버퍼 잔류로 무한 루프

■ 해결

`scanf` 반환값 확인 + `while(getchar() != '\n')` 버퍼 비우기

■ 핵심

어떤 잘못된 입력에도 안정적 동작 보장

게임 규칙

총 5턴의 운영을 마치면 네 지표를 종합해 최종 등급이 결정됩니다. 단, 위기 상황을 넘기지 못하면 즉시 종료됩니다.

▶ PROGRESSION · 진행 규칙

- 1 플레이어 이름 입력 후 게임 시작
- 2 총 5턴 동안 랜덤 이벤트 발생
- 3 매 턴 1 / 2 / 3번 중 선택
- 4 선택 결과에 따라 4대 지표 즉시 변동
- 5 5턴 클리어 시 최종 등급 산출

FAIL CONDITIONS 실패 조건

- 도난 위험도 ≥ 100
- 평판 ≤ 0

GRADE SYSTEM 등급 기준

- ★ S ≥ 200 점
- A ≥ 150 점
- B ≥ 100 점
- C < 100 점

점수 = 보안 + 매출 + 평판 - 위험도

실행 결과

●●● MAIN MENU

=====

[무인편의점 관리자]

=====

1. 게임 시작

2. 게임 설명

3. 최고 점수 확인

4. 종료

=====

메뉴 선택: 1

플레이어 이름을 입력하세요: 박수용

●●● EVENT RESPONSE

[1번째 상황 / 5턴]

[상황] 손님이 상품을 들고 결제하지 않은 채 출구로 이동 중입니다.

1. 바로 경찰에 신고한다.

2. 경고 방송을 한다. ← 선택

3. 조금 더 지켜본다.

→ 적절한 경고로 상황을 해결하였습니다.

●●● FINAL RESULT

=====

[최종 결과]

=====

보안 점수 : 85 매출 : 120

평판 : 65 위험 : 10

=====

최종 점수 : 260

등급: ★ S등급 ★ 매우 우수한 관리자!

=====

[저장] 새로운 최고 점수! (260점)

실행 결과

입력 검증 & 최고 점수

▶ 잘못된 입력 처리 화면

●●● INPUT VALIDATION

선택 (1/2/3): a
[오류] 숫자만 입력하세요.

선택 (1/2/3): 5
[오류] 1~3만 선택 가능.

선택 (1/2/3): 2
→ 정상 처리

- 문자 입력 시 즉시 오류 안내 후 재입력
- 범위를 벗어난 숫자(5)도 차단
- 올바른 값(1~3)만 정상 처리

▶ 최고 점수 확인 화면

●●● HIGH SCORE

=====

[최고 점수]

=====

플레이어 : 박수용
점수 : 260점

=====

- highscore.txt에서 최고 기록을 불러옴
- 플레이어 이름과 점수를 함께 표시
- 새 기록 달성 시 자동으로 갱신

결과 및 결론

10

구현 이벤트 시나리오

8

적용 C언어 핵심 기술

4

실시간 상태 지표

0(n)

Fisher-Yates 셔플

🏆 프로젝트 성과

- ✓ 선택 기반 시뮬레이션을 완성도 있게 구현
- ✓ 4대 지표 균형 설계로 반복 플레이 유도
- ✓ 셔플로 매 게임 다른 전개 보장

📦 학습 성과

- ✓ 구조체·포인터로 상태를 체계적으로 관리
- ✓ 알고리즘을 실제 기능에 적용하는 경험
- ✓ 모듈 단위로 기능을 나누는 설계 습득

🔧 핵심 기술 역량

- ✓ 배열·반복·조건 기반 게임 루프 설계
- ✓ 난수·파일 입출력으로 데이터 처리
- ✓ 매크로로 유지보수 쉬운 코드 구성

감사합니다 · Q&A